

State Management and Optimization

Objectives

- ◆ Define and describe state management
- ◆ Explain and describe how to use cookies and application and session state
- ◆ Explain how to use caches to improve performance of an application
- ◆ Explain the process of bundling and minification

State Management

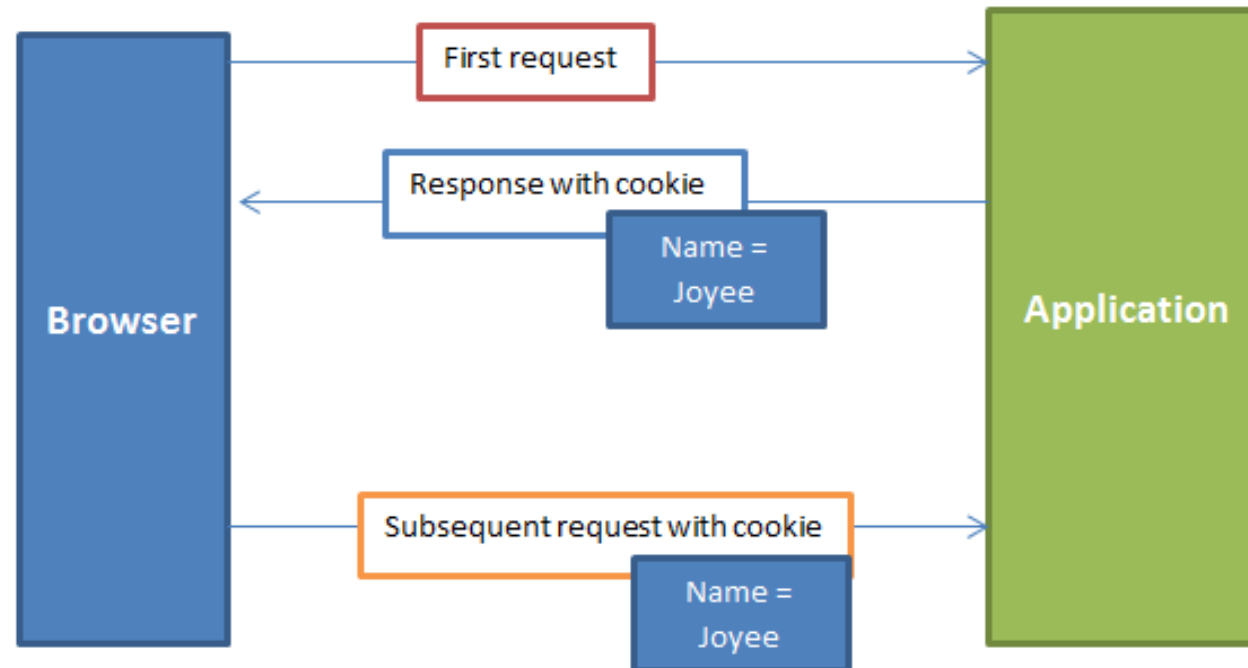
- ◆ Web pages use HTTP protocol to communicate between a Web browser and a Webserver.
- ◆ HTTP is a stateless protocol and cannot automatically indicate whether the sequential requests are coming from the same or different clients.
- ◆ You can implement state management in an ASP.NET MVC application using one of the following options:
 - ◆ Cookies
 - ◆ TempData
 - ◆ Application State
 - ◆ Session State

Cookies

- ◆ Cookies are used to store small pieces of information related to a user's computer, such as its IP address, browser type, operating system, and Web pages last visited.
- ◆ The purpose of storing this information is to offer a personalized experience to the user.
- ◆ Cookies are sent to a client computer along with the page output.
- ◆ These cookies are stored on the client's computer.
- ◆ When a browser requests the same page the next time, it sends the cookie along with the request information.
- ◆ The Web server reads the cookie and extracts its value.
- ◆ It then, process the Web page according to the information contained in the cookie and renders it on the Web browser.

Cookies

- ◆ There are two types of cookies:
 - ◆ The first one is the session cookies that are stored in the browser's memory that are transmitted through the header during every request.
 - ◆ The other type of cookie is the persistent cookies that are stored in text files on a user's computer. This type of cookie is useful when you need to store information for a longtime.



- ◆ Following code snippet shows how to create persistent cookies:

```
Response.Cookies ["UserName"].Value = "John";  
Response.Cookies ["UserName"].Expires = DateTime.Now.AddDays(2);  
Response.Cookies ["LastVisited"].Value = DateTime.Now.ToString();  
Response.Cookies ["LastVisited"].Expires = DateTime.Now.AddDays(2);
```

- ◆ This code creates the UserName and LastVisited cookies. The UserName cookie stores the name of a user and the LastVisited cookie stores the date and time when the user last visited the page. The expiry period for both the cookies is set as 2 days.
- ◆ Once you have a cookie, you can access the value of the cookie by using the built-in Request object.
- ◆ On the other hand to modify a cookie, you need to use the built-in Response object.

Cookies

- ◆ Following code snippet shows how to access a cookie that stores a single value

```
if (Request.Cookies["UserName"].Value != null)
{
    string Name = Request.Cookies["UserName"].Value;
}
```

- ◆ Following code snippet shows how to access a cookie that stores multiple values:

```
if (Request.Cookies["UserInfo"] != null)
{
    string Name = Request.Cookies["UserInfo"]["UserName"];
    string VisitedOn = Request.Cookies["UserInfo"]["LastVisited"];
}
```

- ◆ TempData:
 - ◆ Is a dictionary that stores temporary data in the form of key-value pairs.
 - ◆ Allows store values between requests.
- ◆ You can add values to TempData by adding them to TempData collection.
- ◆ The data stored in TempData is available during the current and subsequent requests.
- ◆ TempData is useful in situations when you need to pass data to a view during a page redirect

- ◆ Following code snippet shows the content of a controller class named HomeController

```
public class HomeController : Controller{  
    public ActionResult Index(){  
        TempData["tempText"] = "Welcome to MVC";  
        return RedirectToAction("Browse");  
    }  
    public ActionResult Browse(){  
        return View();  
    }  
}
```

- ◆ In this code, when you send request for the Index() action method, the value, Welcome to MVC, is stored in the TempData key, named tempText. Hence, you are automatically redirected to the Browse() action method that will render a view

TempData

- ◆ Following code snippet shows the content of the Browse() action method:

```
@{var msg = TempData["tempText"] as String;}  
@msg
```

- ◆ The code renders a view that access the data stored for it in TempData and then display it to the user.

Application State

- ◆ Application state enables storing application-specific information as key-value pairs.
- ◆ When a user accesses any URL, application state is created for the first time.
- ◆ After that it stores the application-specific information.
- ◆ This information can be shared with all the pages and user sessions of the application.
- ◆ For this, you need to use the `HttpApplicationState` class.
- ◆ This class is accessed by using the `Application` property of the `HttpContext` object

Application State

- ◆ Whenever any application events occur, the application state is initialized and manipulated.
- ◆ These application events include the following:
 - ◆ `Application.Start`: Event is raised when an application receives a request for a page for the first time, and when the server or the application is restarted. Event handler of this event contains the code for initializing the application variables.
 - ◆ `Application.End`: Event is raised when the server or the application is stopped or restarted. Event handler of this event contains the code to clear the resources that the application has already used.
 - ◆ `Application.Error`: Event is raised when an unhandled error occurs.
- ◆ The handlers for these three types of events are defined in the `Global.asax` file.
- ◆ You can write the code to manipulate the application state in one of these event handlers.

Application State

- ◆ To store application-specific information in application state, you need to create variables and objects.
- ◆ Then, you can add these variables and objects to the application state.
- ◆ These variables and objects are accessible for any components of an ASP.NET MVC application.
- ◆ You need to write the code for initializing these application variables and objects inside the `Application_Start()` function available in the `Global.asax` file.
- ◆ Following code snippet shows creating a variable named `TestVariable` and then, stores it in the application state:

```
HttpContext.Application ["TestVariable"] = "Welcome to MVC";
```

Application State

- ◆ When you run the application that includes the newly created variable, any pages of this application can retrieve the value of this variable.
- ◆ □ Following code snippet shows how to access the value of TestVariable:

```
stringval = (string)HttpContext.Application["TestVariable"];
```

- ◆ This code access the value included in the TestVariable and then, stores it in a local variable named val.
- ◆ Once you have created and added a variable or object to an application state, you can also remove it from the application state using the Remove() method.

Application State

- ◆ Following code snippet shows how to remove an application variable named, TestVariable, from the application state

```
HttpContext.Application.Remove("TestVariable");
```

- ◆ This code uses the Remove() method that will remove the variable named, TestVariable from the application state.
- ◆ You can also use the RemoveAll() method to remove all the available variables present in the application state.
- ◆ Following code snippet shows using the RemoveAll() method:

```
HttpContext.Application.RemoveAll();
```

- ◆ A session state stores session-specific information for an ASP.NET MVC application.
- ◆ However, the scope of session state is limited to the current browser session.
- ◆ When many users access an application simultaneously, then, each of these users will have a different session state.
- ◆ Similar to application state, a session state stores application-specific data in key-value pairs.
- ◆ These session-specific information should be maintained between server round trips and requests for pages.

- ◆ An active session is identified and tracked by a unique session ID string containing American Standard Code for Information Interchange (ASCII) characters.
- ◆ The ASP.NET MVC Framework provides the SessionStateModule class that enables you to generate and obtain the session ID strings.
- ◆ Like application state, you can also store objects and variables in a session state.
- ◆ However, a session variable remains active for 20 minutes without any user interaction.
- ◆ You can use the same methods as application state, to add and access variable or objects from the session state.

- ◆ Following code snippet shows adding a variable named, TestVariable in a session state:

```
Session["TestVariable"]="Welcome to MVC Application";
```

- ◆ Following code snippet shows how to access the value of the newly created variable

```
stringval = (string) Session["TestVariable"];
```

- ◆ While using session state, you need to consider the following issues:
 - ◆ A variable or an object that is added to a session state remains until a user closes the browser window. By default, a variable or object is automatically removed from the session state after 20 minutes if a user does not request a page.
 - ◆ A variable or an object added to the session state is related to a particular user.
- ◆ Similar to application state, a session state can be globally accessed by the current user.
- ◆ A session state can be lost in case of the following situations:
 - ◆ User closes or restarts the browser.
 - ◆ User accesses the same Web page through a different browser window.
 - ◆ The session times out because of inactivity.
 - ◆ The `Session.Abandon()` method is called within the page code.

- ◆ To remove an object or a variable that has been added to a session state, you can use the Remove() or RemoveAll() methods.
- ◆ While using session states in an application, you need to configure it in the Web.config file of the application.
- ◆ The Web.config file enables you to define advanced options, such as the timeout and the session state mode.
- ◆ Following code snippet shows the options that you can configure inside the <sessionState> element:

```
<sessionState  
cookieless="UseCookies" cookieName="Test_SessionID"  
timeout="20"  
mode="InProc" />
```

- ◆ You can use the cookieless attribute to specify whether a cookie is to be used or not.
- ◆ Following table lists the values that you can specify for the cookieless attribute:

Value	Description
UseCookies	Allows you to specify that cookies are always used.
UseUri	Allows you to specify that cookies are never used.
UseDeviceProfile	Allows you to specify that the application should check the Browser Capabilities object to identify whether to use cookies.
AutoDetect	Allows you to specify that cookies should be used if the browser supports them.

- ◆ You can use the timeout attribute of session state to specify a time period in minutes.
- ◆ You can use the mode attribute to specify where the values of the session state is to be stored.
- ◆ You can use one of the following values of the mode attribute:
 - ◆ Custom: Allows you to specify that a custom data store should be used to store the session-state data.
 - ◆ InProc: Allows you to specify that the data is to be stored in the same process as the application. The Session.End event is raised only in this mode.
 - ◆ Off: Allows you to specify that the session state is disabled.
 - ◆ SQLServer: Allows you to specify that the session state is to be stored by using SQL Server database to store the state data.
 - ◆ StateServer: Allows you to specify that the session state is to be stored in a service running on the server or a dedicated server.